

بسمه تعالی

بررسی فایل سیستم‌های مناسب برای لاگ‌گیری داده در میکروکنترلرهای STM32

شرکت فاتح آسمان شریف

تهیه کننده: رضا سعیدی

مرداد ماه ۱۴۰۴

فهرست

مقدمه	3
معیار های ارزیابی فایل سیستم	4
بررسی فایل سیستم ها	5
◦ FatFS	5
◦ ExFAT	6
◦ LittleFS	7
◦ Ext3/Ext4 (lwest4)	8
◦ more ...	9
مقایسه فایل سیستم ها	11
مفاهیم کلیدی و اطلاعات تکمیلی	13
منابع	15

ثبت داده‌های سنسور، تصاویر و یا نتایج پردازشی در بسیاری از سیستم‌های نهفته نیازمند ذخیره‌سازی روی کارت‌های SD است.

در بسیاری از کاربردهای صنعتی و جاسازی‌شده (Embedded) نیاز به ثبت داده‌ها با سرعت بالا و به صورت پایدار بر روی حافظه‌های جانبی مانند کارت SD وجود دارد.

برای ثبت داده‌ها با سرعت بالا روی کارت SD در میکروکنترلرهای STM32، معمولاً از FatFS (فایل سیستم FAT32/FAT16) استفاده می‌شود. اما در عمل FAT32 در برابر **قطع ناگهانی برق** و نویزهای محیطی مقاوم نیست. طبق بررسی‌های انجام‌شده (مانند مقاله هکادی)، فایل سیستم‌های رایج مانند FAT32 به خوبی تغییرات ناگهانی برق را تحمل نمی‌کنند و در صورت قطع ناگهانی برق حین نوشتن، ممکن است بخش‌هایی از فایل از دست برود یا خود ساختار فایل سیستم آلوده شود. در مقابل، فایل سیستم‌های ژورنال‌شده (journaling) مانند ext3/4، NTFS یا LittleFS تغییرات را ابتدا در یک «ژورنال» ثبت می‌کنند و سپس به حافظه می‌نویسند. این مکانیسم تضمین می‌کند که در صورت قطع برق، فایل سیستم به حالت پایدار قبلی برگردد و احتمال فساد کلی کاهش یابد.

- **سرعت نوشتن** : با استفاده از رابط SDIO و DMA میتوان به بالاترین سرعت دست یافت . سرعت SPI معمولاً کمتر است . به همین دلیل موضوع سرعت نوشتن داده بر روی SD Card از اهمیت بالایی برخوردار است و یکی از مهم ترین معیار های ارزیابی فایل سیستم ها به شمار می آید .
- **مقاومت در برابر قطعی برق** : فایل سیستم باید پس از قطع ناگهانی برق، داده های قبلاً نوشته شده را حفظ کند و فساد ناگهانی را به حداقل برساند. سیستم های ژورنال شده (مثلاً ext3/4 ، LittleFS) در این زمینه قوی تر عمل می کنند.
- **مصرف RAM و Flash** : هر میکروکنترلر STM32 محدودیت RAM و فلش خاص خود را دارد. مثلاً STM32F4 حدود 192 KB RAM و 1MB Flash دارد، ولی مدل های F1/F0 بسیار کمتر هستند . پس در انتخاب فایل سیستم باید به مصرف حافظه نگاه کرد.
- **سازگاری با سیستم های عامل** : چون احتمال دارد پس از ذخیره، کارت SD به کامپیوتر متصل و اطلاعات استخراج شود (مثلاً عکس های دوربین)، استفاده از فایل سیستمی که توسط سیستم عامل های رایج شناسایی می شود مهم است . FAT32 و exFAT روی ویندوز و لینوکس پشتیبانی می شوند؛ ext4 روی لینوکس پشتیبانی است اما روی ویندوز معمولاً نیاز به درایور دارد . LittleFS یا سیستم های سفارشی نیاز به ابزار مخصوص برای خواندن در کامپیوتر دارند (مثلاً مدیا فایل LittleFS-FUSE) که بهره برداری را پیچیده می کند.
- **سادگی پیاده سازی** : منابع رایگان و متن باز مطلوب اند . به طور مثال FATFS که توسط CHAN توسعه پیدا کرده و همچنین LittleFS متن باز هستند .

در این بخش برآنیم که چند سیستم فایل مطرح و قابل استفاده در پروژه ها را بیان کنیم و نقاط قوت و ضعف هر کدام را بررسی کنیم تا نهایتاً باتوجه به نیاز های موجود ، فایل سیستم متناسب را انتخاب کنیم:

FATFS ○

کتابخانه FatFs پیاده سازی سبک و متن بازی از سیستم فایل FAT32 ارائه می کند که مبتنی بر جدول Table Allocation است. داده ها در خوشه های 32KB ذخیره می شوند و متادیتا در ساختار دایرکتوری نگهداری می شود. برای پیاده سازی در STM32 ، فایل های ff.c ، gff.h و diskio.c به پروژه اضافه شده و توابع disk_initialize ، disk_read و disk_write با HAL_SD پر می شوند. پس از فراخوانی f_mount ، می توان با f_open ، f_write و در نهایت f_sync داده را نوشت و فایل را بست.

مزایا : FAT32 فراگیرترین و استانداردترین فرمت روی کارت های SD است. کتابخانه FatFs به صورت متن باز در دسترس است و روی STM32 به راحتی پیاده سازی می شود. اندازه کد FatFs نسبتاً کوچک است (مثلاً چند کیلوبایت با پیکربندی های بهینه) و سرعت نوشتن در این سیستم عامل سریع است اگر داده را به صورت بلوک های بزرگ و پیوسته بنویسید. همچنین ابزارهای ویندوز/لینوکس برای خواندن FAT32 در دسترس است.

معایب : FAT32 فاقد مکانیزم ژورنال است؛ در نتیجه اگر در حین نوشتن برق قطع شود، ساختار FAT یا دایرکتوری ممکن است خراب شود. مطالعات نشان می دهد که در صورت قطع برق ناگهانی، تنها انتهای فایل می تواند ناقص ثبت شود ولی گاهی کل فایل سیستم خراب می شود. در واقع در سناریوی قطع برق ناگهانی برای کم کردن خطر باید همواره f_sync() فراخوانی کرد و داده ها را در حافظه کش نگه داری کرد، اما حتی این کار تضمین کاملی نیست. علاوه بر این، به روزرسانی متادیتا

(مانند زمان تغییر فایل) می‌تواند دو برابر نوشتن بلوک‌ها را افزایش دهد. همچنین FAT32 محدودیت حجم فایل 4GB دارد و ردیف‌بندی دایرکتوری (بدون اندیس) در حجم فایل‌های بسیار بزرگ کند می‌شود. در کل، FAT32 سرعت خوبی دارد اما در مقابل قطع ناگهانی برق آسیب‌پذیر است (بارها مشاهده شده که کارت‌ها به «خاموشی غیرمنتظره» واکنش نشان داده و نیاز به فرمت پیدا می‌کنند)

بهبودها: برخی محیط‌های تجاری افزونه‌ی ژورنال دارند. مثلاً کیت *File System Component* در Keil یک مکانیزم اختصاصی ژورنال برای FAT ارائه می‌دهد که «عملیات فایل I/O را در مقابل کرش یا قطع برق مقاوم می‌کند» به طوری که فایل سیستم پس از بازیابی به وضعیت سالم قبلی بازمی‌گردد، اما این افزونه تجاری است و در پروژه‌های متن‌باز رایگان در دسترس نیست.

EXFAT ○

این فایل سیستم برای حذف محدودیت 4 GB فایل در FAT32 طراحی شده و خوشه‌های بزرگتری (32 – 128 KB) به کار می‌برد. ساختار exFAT با الگوریتم تخصیص سریع‌تر، کارایی خوبی برای فایل‌های حجیم ارائه می‌دهد. با این حال، exFAT نیز مانند FAT32 بدون مکانیزم ژورنال عمل می‌کند. پیاده‌سازی متن‌باز آن محدود است و حجم کد آن حدود 10 – 15KB است و برای استفاده کامل نیاز به مجوز مایکروسافت دارد. سرعت نوشتن آن غالباً مشابه FAT32 است، اما در پروژه‌هایی که با فایل‌های چند گیگابایتی سروکار دارند، exFAT گزینه‌ای مناسب است.

مزایا: برای کارت‌های با ظرفیت بسیار بالا و فایل‌های بزرگ (بزرگتر از 4 GB) فرمت exFAT استفاده می‌شود. FatFs نیز (در برخی نسخه‌ها) پشتیبانی از exFAT را دارد. روی ویندوز و لینوکس مدرن بدون مشکل mount می‌شود.

معایب: exFAT نیز مکانیزم ژورنال ندارد و در برابر قطع برق بسیار به FAT32 شباهت دارد. ضمن آن‌که فرمت exFAT تحت امتیاز مایکروسافت است و استفاده تجاری از آن نیاز به مجوز دارد. پیچیدگی و سربار بیشتری نسبت به FAT32 دارد و سرعت آن مشابه FAT32 است. در سناریوی حساس به قطع برق، exFAT تغییری ایجاد نمی‌کند و در بهترین حالت مانند FAT عمل می‌کند.

این فایل سیستم یک فایل سیستم تعبیه شده مدرن است که برای حافظه های Flash طراحی شده و به خوبی روی کارت های SD نیز کار می کند. این سیستم با استفاده از مکانیزم copy-on-write برای متادیتا و تضمین atomic write ، تضمین می کند که ساختار فایل سیستم هرگز خراب نشود.

مزایا : LittleFS یک فایل سیستم متن باز توسعه ی ARM برای کاربردهای نهفته است که برای محافظت از داده در برابر قطع برق تصادفی طراحی شده . این فایل سیستم از روش *copy-on-write* برای به روزرسانی ساختارهای درونی استفاده می کند؛ بدین معنی که همه عملیات فایل (ایجاد/نوشتن) ابتدا در ناحیه ای موقت اعمال شده و در نهایت به حافظه اصلی منتقل می شود. در صورت قطع برق حین نوشتن، این الگوریتم تضمین می کند که فایل سیستم به آخرین حالت پایدار بازمی گردد و تنها آخرین نوشتن ناقص حذف می شود . بنابراین LittleFS به طور ذاتی مقاومت خوبی در برابر خطای قطع برق دارد. همچنین LittleFS دارای تراکم کد اندک و استفاده کم از RAM است؛ به عنوان نمونه، پیاده سازی LittleFS در ESP32 حدود ۱۴ کیلوبایت کد و تنها ۱۰۰ بایت RAM هنگام بارگذاری فایل سیستم مصرف می کند که برای میکروکنترلرهایی مانند STM32F4 کاملاً مناسب است .

LittleFS پشتیبانی از دایرکتوری واقعی و نام فایل های بلند (تا ۲۵۴ کاراکتر UTF-8) را نیز دارد که برای پروژه های پیچیده تر مفید است. علاوه بر این، LittleFS مکانیزم *wear leveling* داخلی دارد که عمر تراشه های فلش را افزایش می دهد (اگرچه در کارت SD ، خود کنترلر کارت این کار را انجام می دهد)

معایب : مهم ترین ضعف LittleFS بهینه نبودن سرعت برای فایل های بزرگ و تعداد زیاد عملیات I/O است. چون LittleFS دائماً ساختارهای درونی را بررسی و به روزرسانی می کند، عملکرد آن در نوشتن پیوسته داده کمتر از FAT است. به طور پیش فرض LittleFS مقدار کمی RAM تخصیص می دهد تا امن و سبک باقی بماند؛ در نتیجه اگر رم بیشتری در دسترس است، می توان با افزایش اندازه ی کش و پارامترهای حافظه سرعت آن را بهبود بخشید . در پیاده سازی های معمول، LittleFS سریع ترین گزینه نیست، اما پایداری آن در برابر خطا بسیار بالاست.

نکته منفی دیگر این است که کارت SD با LittleFS را به صورت مستقیم نمی توان روی ویندوز/لینوکس mount کرد؛ برای خواندن فایل ها در کامپیوتر نیاز به درایور یا ابزار کمکی (مثلاً پروژه FUSE از LittleFS) است .

کاربرد مناسب: LittleFS برای میکروکنترلرهای سری F4/F7/H7 با سرعت بالا و منابع RAM معقول گزینه ای عالی است، به ویژه وقتی پایداری داده در اولویت قرار دارد. برای میکروهای بسیار کوچک (مثل STM32F0/F1 با RAM بسیار کم) نیز LittleFS ممکن است سنگین باشد، اما با استفاده از پیکربندی بهینه می تواند کار کند. پیاده سازی متن باز آن (گیت هاب ARM) نیز به راحتی در پروژه های FreeRTOS یا بدون RTOS قابل استفاده است.

○ قابل ذکر است که LittleFS عملکرد بهینه ای در بلوک های 512 تا 4096 بایت دارد ولی در مقابل FatFS بهترین عملکرد در بلوک های 4KB تا 32KB را داراست .

○ EXT3/EXT4 (LWEXT4)

فایل سیستم های ext3 و ext4 از ساختار inode-based و journaling برای متادیتا استفاده می کنند . lwext4 نسخه ای سبک برای میکروکنترلرهاست که امکان mount و عملیات I/O شبیه لینوکس را فراهم می کند. پیاده سازی شامل راه اندازی لایه block device با HAL_SD و mount با lwext4_mount است. این گزینه با مصرف 25KB Flash و 32KB RAM – 16، سرعت 5-8 MB/s و پایداری بالا برای پروژه های با ساختار دایرکتوری پیچیده مناسب است.

مزایا: فایل سیستم های لینوکسی ext3 و ext4 کاملاً ژورنال شده هستند. پروژه lwext4 (بیل گوستکا) یک پیاده سازی سبک ext2/3/4 برای میکروکنترلرهاست. ویژگی های کلیدی lwext4 شامل شاخص گذاری دایرکتوری (پیمایش سریع فایل ها)، extent ها (برای کارایی در فایل های بزرگ) و ژورنال داخلی با بازیابی پس از قطع برق است، این یعنی ساختارهای فایل سیستم ext4 می توانند به طور ناپیوسته روزرسانی شوند و پس از قطع برق، به حالت قبلی بازگردند. همچنین lwext4 برای

حافظه‌های فلش با تراشه پوشش داده شده مناسب است (مانند کارت SD که خود سطح پوشش فرسایش دارد)

معایب: lwext4 نیازمند کد بیشتری است. طبق مستندات رسمی، تنها پشتیبانی ext2 حدود ۲۰ کیلوبایت کد در بخش text. اشغال می‌کند و فعال‌سازی تمام امکانات ext4 تا حدود 50 KB کد می‌برد (علاوه بر چند کیلوبایت داده برای کش و ژورنال). این برای میکروکنترلرهای قوی مثل STM32F7/H7 قابل قبول است، ولی برای سری‌های ضعیف‌تر (F1/F0) سنگین است. همچنین lwext4 به malloc (حافظه دینامیک) نیاز دارد و پیچیدگی‌هایی مانند مدیریت کش و تنظیمات مختلف (ext2/3/4) دارد. از سوی دیگر، داده‌های ذخیره‌شده به این شکل عیناً توسط ویندوز قابل خواندن نیست مگر از طریق سیستم‌های فایل ext. اما اگر قرار باشد کارت به لینوکس متصل شود، ext4 مزیت زیادی دارد.

در مجموع، lwext4 یک گزینه‌ی متن‌باز بسیار قدرتمند است اما صرفاً برای پروژه‌هایی که به امکانات پیشرفته (مانند ژورنال اختصاصی، پرونده‌های بزرگ و تعداد زیاد فایل) نیاز دارند و میکروکنترلر قدرتمندی دارند مناسب است.

MORE ... ○

SPIFFS : یک فایل سیستم قدیمی مخصوص حافظه‌های SPI فلش است که در پروژه‌های قدیمی‌ای اس‌پی استفاده می‌شد. SPIFFS نه ژورنال دارد و نه بهینه برای سرعت بالا؛ عمدتاً جای خود را به LittleFS داده است. SPIFFS تقریباً منسوخ شده است و پیشنهاد نمی‌شود. (برخی گفته‌اند عملکرد SPIFFS از LittleFS بسیار کندتر است و پشتیبانی آن رو به پایان است).

سیستم فایل‌های اختصاصی بدون FAT: اگر تعداد جریان‌های داده معلوم باشد، می‌توان بدون فایل سیستم سنتی عمل کرد. به عنوان مثال، یک دستگاه می‌تواند داده‌ها را در فایل باینری بزرگ واحد به ترتیب بنویسد و هر بلوک را با برچسب‌هایی نشانه‌گذاری کند. این روش سرعت بسیار بالایی دارد و

امكان بازيابى ساده با بررسى بلوكها را فراهم مى كند. اما در اين حالت نمى توان كارت را مستقيماً در يك كامپيوتر عمومى خواند مگر اينكه نرم افزار خواننده اختصاصى نوشته شود.

FAT تجارى با ژورنال: برخى شركتها مثل SEGGER emFile فايل سيستم FAT با افزونه ژورنال ارائه مى دهند. مثلاً مستند Keil نشان مى دهد كه *File System Component* اين شركت مى تواند ژورنال را فعال كند تا تغييرات ابتدا ثبت شوند و فايل سيستم پس از حادثه، خراب نشود، اما اين محصولات معمولاً تجارى اند و براى پروژه هاى متن باز رايجان نيستند.

مقایسه فایل سیستم ها

در جدول زیر یک سری پارامتر های مهم در انجام پروژه ها را برای فایل سیستم های مختلف مقایسه میکنیم :

پارامتر	FatFS / exFAT	LittleFS	Lwext4 (ext3/ext4)
کد (Flash)	6 – 12KB	1 – 4KB	journal/extends با 50KB
حافظه RAM	1 – 2KB	1KB قابل پیگر بندی	journal/extends با 12KB تا
حداکثر حجم فایل / پارتیشن	4GB تا FAT32 64ZB تا exFAT ($10^{16} \times 4$)	2GB وابسته به پیگر بندی	ext4 کامل : 16TB
سرعت نوشتن و خواندن با رابط SDIO(STM32F4)	نوشتن : 8.2 – 10.5MB/s خواندن : 12 – 15MB/s	نوشتن : 4.8 – 6.3MB/s خواندن : 6.5 – 8MB/s	نوشتن : 6 – 7.5MB/s خواندن : 8.5 – 10.5MB/s
مقاومت در برابر قطعی برق	ندارد	بسیار بالا	بالاترین سطح
خوانایی روی PC	فوق العاده؛ FAT32/exFAT روی ویندوز، لینوکس	ندارد؛ باید با ابزاری مثل fuse-littlefs به صورت فایل ایمیج روی PC قابل خواندن شود	ext4 در لینوکس بدون مشکل؛ در ویندوز نیاز به درایور
سازگاری با STM32	STM32F0/F1/F3/F4/F7/H7 STM32H7 فقط ecFAT	STM32F0/F1/F3/F4/L4/H7	STM32F7/H7

بهترین گزینه برای هر پارامتر :

- ✓ مدت زمان اجرا و حجم کد (Flash) : فایل سیستم های FatFS و LittleFS هر دو سبک‌اند؛ lwext4 حدود ۴-۵ برابر سنگین‌تر است.
- ✓ مصرف حافظه RAM : فایل سیستم LittleFS کمترین مصرف را دارد (حدود ۱ کیلوبایت)، بعد FatFS در حالت tiny (۰.۶-۱ کیلوبایت) و در سخت‌گیرانه‌ترین حالت lwext4 (۸ تا ۱۳ کیلوبایت)
- ✓ سرعت نوشتن بالا : FatFS/exFAT با SDIO و DMA بهترین سرعت را دارد
- ✓ مقاومت در برابر قطعی برق: LittleFS بسیار عالی‌ست، lwext4 نیز با journaling تضمین بالایی دارد، ولی FatFS در صورت قطع ناگهانی ممکن است خراب شود.
- ✓ خوانایی و انتقال داده به کامپیوتر : FatFS/exFAT کاملاً قابل استفاده است؛ lwext4 لاینوکس عالی، روی ویندوز محدود است؛ LittleFS نیاز به ابزار تبدیلی دارد.

Atomic write / journaling

در فایل سیستم‌هایی مانند ext3/ext4 (lwest4)، سیستم journaling به این صورت عمل می‌کند که تغییرات متادیتا ابتدا در ناحیه‌ی مخصوص ژورنال نوشته می‌شوند و پس از اطمینان از صحت آن، در ساختار اصلی فایل سیستم اعمال می‌گردند. در صورت خاموشی ناگهانی، سیستم با استفاده از ژورنال قادر است به آخرین وضعیت پایدار برگردد. این موضوع به‌ویژه برای کاربردهای حیاتی که قطعی برق محتمل است، بسیار مهم است.

در LittleFS، مشابه مفهوم atomic write وجود دارد، اما بدون journaling سنگین تر. هر بار که داده یا metadata نوشته می‌شود، نسخه قبلی تا اطمینان از صحت نگه‌داشته می‌شود. اگر checksum برابر نباشد، سیستم به‌طور خودکار به نسخه قبلی بازمی‌گردد و filesystem همچنان سالم باقی می‌ماند.

LittleFS در Copy-on-Write و Power-Loss Resilience

LittleFS طوری طراحی شده که حتی در صورت قطع برق در لحظه نوشتن، فایل سیستم خراب نشود. این ویژگی با استفاده از ساختار metadata pairing و copy-on-write محقق می‌شود: metadata در بلوک‌های دوبخشی ذخیره و با checksum مدیریت می‌شوند، و داده‌های فایل ابتدا در مکانی موقت نوشته می‌شوند و سپس جایگزین داده قبلی می‌گردند. ARM اعلام کرده LittleFS در مقایسه با FAT، حدود ۱۳ کیلوبایت کمتر ROM و ۴ کیلوبایت کمتر RAM مصرف می‌کند.

Wear Leveling و مدیریت ماندگاری Flash

LittleFS با استفاده از الگوریتم wear-leveling داخلی، داده را در بلوک‌های مختلف پخش می‌کند تا فرسایش یک ناحیه خاص کنترل شود. این موضوع به‌ویژه در حافظه‌های NOR flash یا SD کارت‌هایی که controller خارجی wear-leveling ندارند، حیاتی است.

حافظه محدود و طراحی بدون recursion

کد LittleFS به گونه‌ای طراحی شده که بدون استفاده از recursion یا منابع نامحدود، مصرف RAM آن محدود و قابل تنظیم باقی بماند. استفاده از buffer هایی با اندازه قابل تعیین باعث می‌شود حافظه مصرفی ثابت باشد و با رشد فایل سیستم افزایش نیابد

ساختار ext4/lwext4 برای MCU

پیاده‌سازی lwext4 برای میکروکنترلر به صورت کاملاً با ویژگی‌های ext3/ext4 مثل indexing دایرکتوری، extents برای فایل‌های بزرگ، journaling، بازیابی خودکار و پشتیبانی از حجم بالا طراحی شده است. نسخه ساده ext2 در حدود ۲۰ کیلوبایت Flash و نسخه کامل ext4 با extents و journal حدود ۴۹ کیلوبایت Flash در Cortex-M4 مصرف دارد. هم به ۸-۱۲ کیلوبایت نیازمند است

سایر مفاهیم ضروری

- **Wear-Leveling**: مدیریت توزیع متعادل نوشتن روی بلوک‌های حافظه برای جلوگیری از تخریب زودرس
- **Atomic Commit**: ترکیبی از copy-on-write و metadata pairs برای نوشتن اتمی.
- **Extent Allocation**: راهکاری در lwext4 برای تخصیص بلوک‌های بزرگ به فایل برای تسریع خواندن و نوشتن.
- **Bounded RAM/ROM**: طراحی LittleFS به گونه‌ای که حافظه مصرفی ثابت بماند حتی با افزایش حجم داده.

۱. Jan Jongboom, «LittleFS – A high-integrity embedded file system», Mbed blog, ARM, 2018 (os.mbed.com)
۲. LittleFS GitHub Repository (design, power-cut resilience, bounded RAM) ([GitHub](https://github.com))
۳. Vyacheslav Azarov on Mbed forum توضیح copy-on-write در LittleFS (os.mbed.com)
۴. Moddable blog «LittleFS – the Best File System for Embedded Projects» (moddable.com)
۵. wext4 GitHub README، journaling، extents، memory footprint ([GitHub](https://github.com)، shithub.us)
۶. ST Community forum سرعت SDIO و تجربه واقعی FATFS روی STM32F4 (wiki.st.com)
۷. Elm-Chan FatFs Appnote و مستندات ESP-IDF file system guide (docs.espressif.com)
۸. [مقاله هاکیدی](#)
۹. اطلاعات و داده های [سایت رسمی ST](#)